TABLE OF CONTENTS

Sr. No.	Content	Page No.
1	**Introduction** 1.1 Background 1.2 Problem Statement 1.3 Objectives	
2	**Frontend Development** 2.1 UI/UX Design 2.2 Components and Structure 2.3 Routing and Navigation 2.4 Responsiveness	
3	**Backend Development** 3.1 Spring Boot Architecture 3.2 REST API Design 3.3 Business Logic Implementation 3.4 Database Configuration 3.5 CRUD Operations Implementation 3.5.1 Create Operation 3.5.2 Read Operation 3.5.3 Update Operation 3.5.4 Delete Operation	
4	**Authentication & Security** 4.1 User Registration and Login 4.2 Validation Techniques 4.3 Security Measures	
5	**Frontend–Backend Integration** 5.1 API Integration (Axios/Fetch) 5.2 Data Flow Mechanism 5.3 Error Handling	
6	**Deployment** 6.1 Deployment Platform 6.2 Deployment Steps 6.3 Live Application URL	
7	**Testing and Validation** 7.1 API Testing (Postman) 7.2 Unit Testing (JUnit/Mockito) 7.3 Test Cases and Results	
8	**Results and Discussion (Screenshots of website)**	
9	**Conclusion**	
10	**References**	

1. **Introduction**

1.1 Background

Parking management has become a major challenge in urban areas due to increasing vehicle density and limited parking spaces. Traditional parking systems rely heavily on manual processes, which are inefficient, time-consuming, and prone to human error. There is a growing need for automated systems that can efficiently manage parking slots, reduce congestion, and provide real-time availability updates. With advancements in web technologies and backend frameworks, it is now possible to build intelligent parking systems that handle booking, tracking, billing, and monitoring seamlessly.

1.2 Problem Statement

Manual parking systems suffer from:

- Lack of real-time slot availability
- Inefficient space utilization
- Manual billing errors
- No proper record of users and bookings
- Security issues due to lack of authentication

The goal is to design a **secure, scalable, and automated Parking Management System** that solves these issues.

1.3 Objectives

- To develop a full-stack web application for parking management
- To implement **user authentication using JWT and Spring Security**
- To manage parking slots using CRUD operations
- To enable users to book and exit parking slots
- To calculate **time-based billing automatically**
- To provide a responsive and interactive dashboard

2. **Frontend Development**

2.1 UI/UX Design

The frontend is designed using **React.js with Vite** and styled using pure CSS.
Key design features:

- Glassmorphism UI
- Dark mode interface
- Color-coded slot indicators (Green: Available, Red: Occupied)
- Smooth animations and transitions

2.2 Components and Structure

Main components:

- Login & Register
- Dashboard
- Admin Slot Management
- Booking History

Each component is modular and reusable.

2.3 Routing and Navigation

Routing is implemented using **React Router**:

- /login → Login page
- /register → Register page
- /dashboard → Main parking view
- /admin → Slot management (Admin only)
- /bookings → Booking history

Protected routes ensure only authorized users can access certain pages.

2.4 Responsiveness

- Fully responsive design
- Works on desktop
- Grid-based layout for slot visualization

3. Backend Development

3.1 Spring Boot Architecture

The backend follows **layered architecture**:

- Controller Layer
- Service Layer
- Repository Layer
- Entity Layer

Technologies used:

- Spring Boot
- Spring Security
- Hibernate/JPA

3.2 REST API Design

Key APIs:

Authentication

- POST `/auth/register`
- POST `/auth/login`

Slots

- GET `/slots`
- POST `/slots`
- PUT `/slots/{id}`
- DELETE `/slots/{id}`

Bookings

- POST `/bookings`
- GET `/bookings`
- POST `/bookings/exit/{id}`
- GET `/bookings/calculate/{id}`

3.3 Business Logic Implementation

- Slot allocation based on availability
- Booking creation with entry time
- Exit process calculates total duration and fee
- Slot status updated dynamically

3.4 Database Configuration

Database: **MySQL**

Tables:

- Users
- ParkingSlots
- Bookings

Connected using:

- Spring Data JPA
- Hibernate ORM

3.5 CRUD Operations Implementation

3.5.1 Create Operation

- Add new users
- Add parking slots
- Create bookings

3.5.2 Read Operation

- Fetch all slots
- Fetch booking history

3.5.3 Update Operation

- Update slot details
- Change slot status

3.5.4 Delete Operation

- Remove parking slots (Admin only)

4. **Authentication & Security**

4.1 User Registration and Login

- Users register with email and password
- Login returns **JWT token**
- Token used for authenticated API access

4.2 Validation Techniques

- Form validation in frontend
- Backend validation using annotations
- Error messages for invalid inputs

4.3 Security Measures

- JWT-based authentication
- Role-based access control (ADMIN, USER)
- Password encryption
- Stateless session handling

5. **Frontend–Backend Integration**

5.1 API Integration (Axios)

- Axios used for API calls
- JWT token stored in local Storage
- Automatically attached to headers

5.2 Data Flow Mechanism

1. User interacts with frontend
2. API request sent via Axios
3. Backend processes request
4. Response returned and displayed

5.3 Error Handling

- Try-catch in backend
- Axios interceptors in frontend
- User-friendly error messages

6. **Deployment**

6.1 Deployment Platform

- Backend: Localhost (Spring Boot server)
- Frontend: Vite development server

6.2 Deployment Steps

1. Start MySQL server
2. Run backend using Maven
3. Run frontend using npm
4. Access via browser

6.3 Live Application URL

(Not deployed – runs locally)

7. **Testing and Validation**

7.1 API Testing (Postman)

- All endpoints tested
- Verified authentication and CRUD operations

7.2 Unit Testing (JUnit/Mockito)

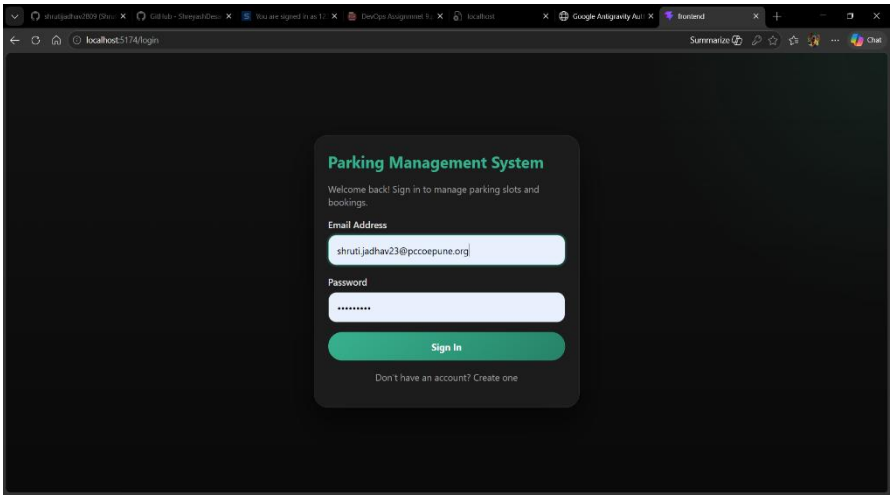
- Basic unit testing for services
- Validated business logic

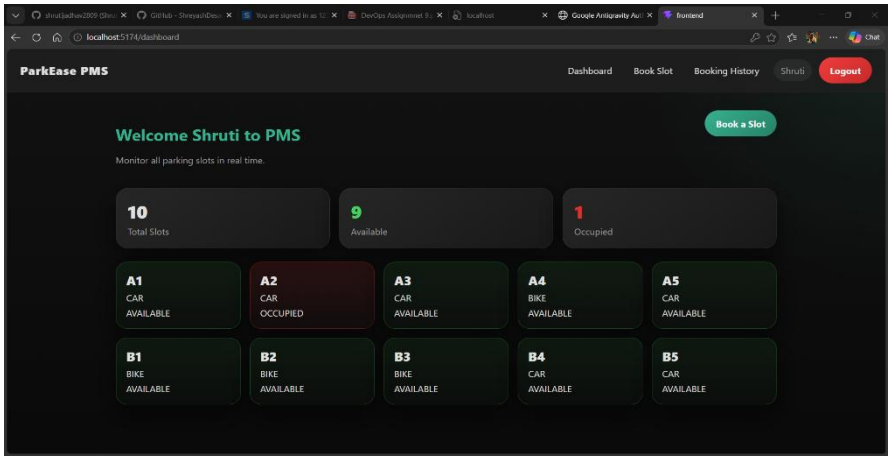
7.3 Test Cases and Results

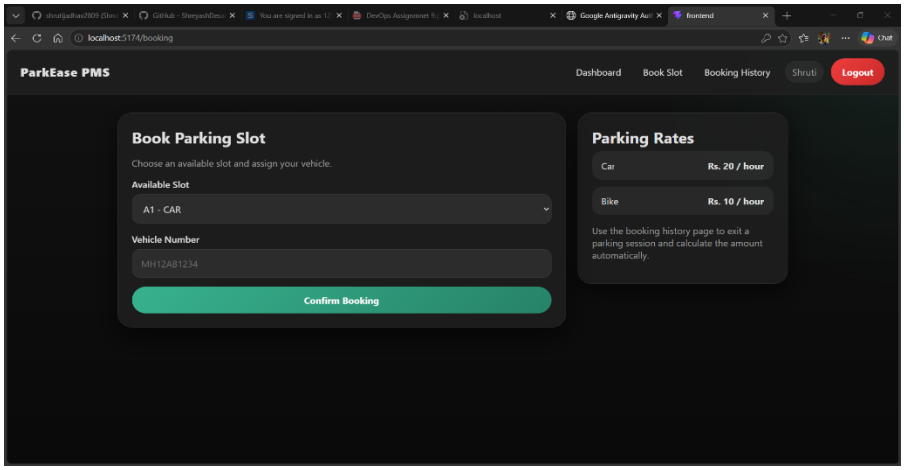
Test Case	Expected Result	Status
User Login	Successful login with JWT	Passed
Slot Booking	Slot becomes occupied	Passed
Exit Booking	Fee calculated	Passed
Admin CRUD	Slot added/updated/deleted	Passed

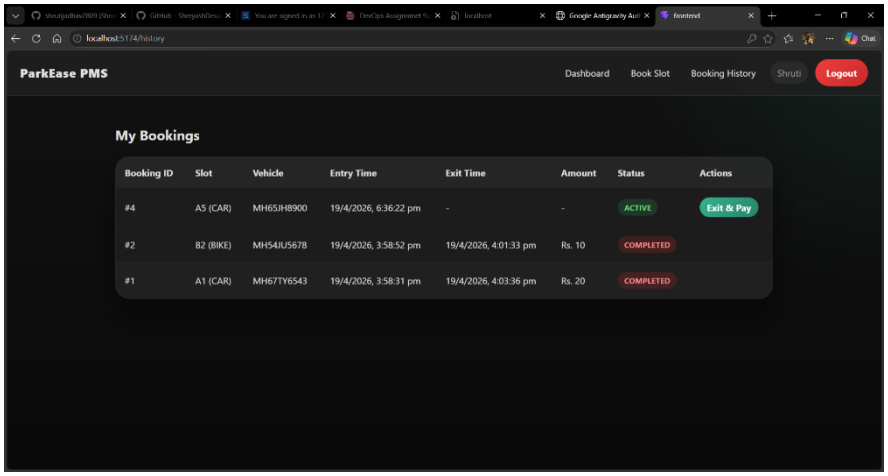
8. Results and Discussion

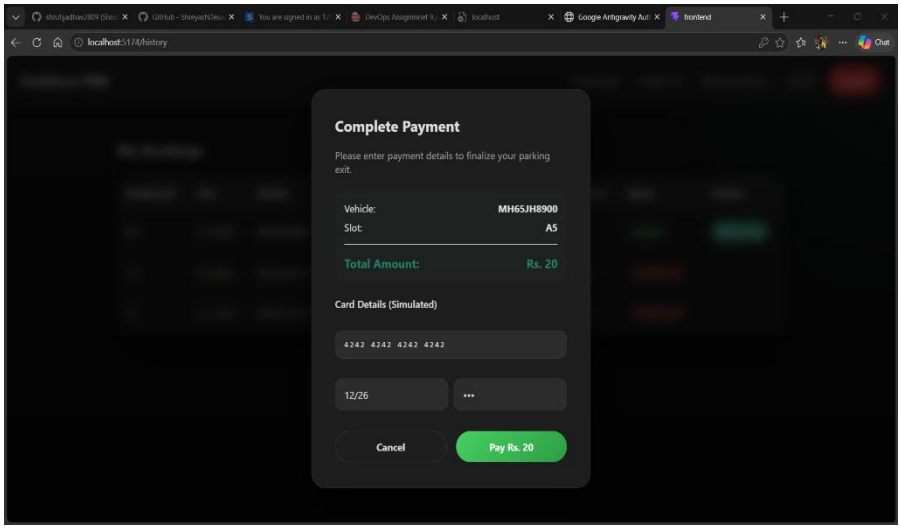
- Successfully implemented full-stack system
- Real-time slot updates visible
- Booking and billing working correctly
- Smooth UI experience

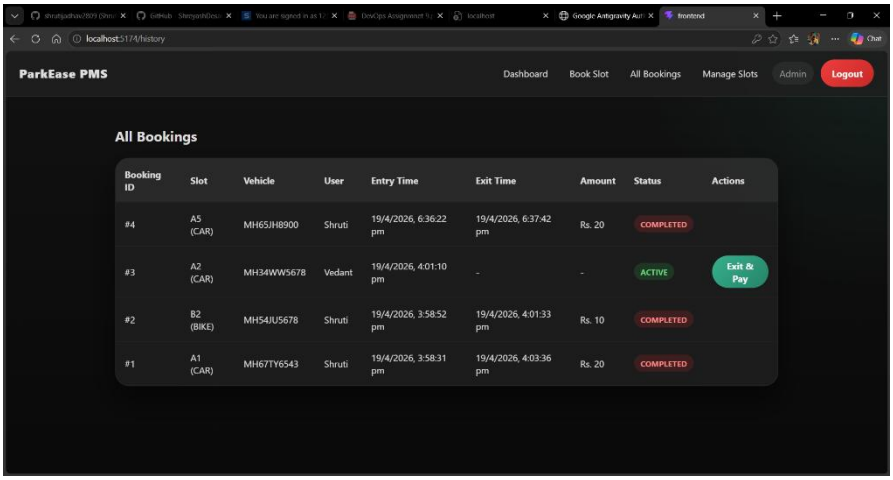


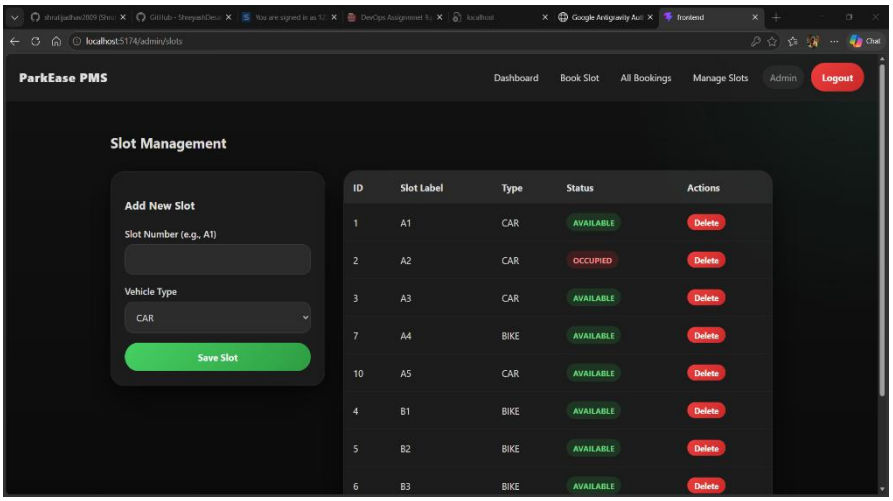



localhost:5174/booking
ParkEase PMS
Dashboard
Book Slot
Booking History
Shruti
Logout
Book Parking Slot
Choose an available slot and assign your vehicle.
Available Slot
A1 - CAR
Vehicle Number
MH12AB1234
Confirm Booking
Parking Rates
Car
Rs. 20 / hour
Bike
Rs. 10 / hour
Use the booking history page to exit a parking session and calculate the amount automatically.


localhost:5174/history
ParkEase PMS
Dashboard
Book Slot
Booking History
Shruti
Logout
My Bookings
Booking ID
Slot
Vehicle
Entry Time
Exit Time
Amount
Status
Actions
#4
A5 (CAR)
MH65JH8900
19/4/2026, 6:36:22 pm
-
-
ACTIVE
Exit & Pay
#2
B2 (BIKE)
MH54JU5678
19/4/2026, 3:58:52 pm
19/4/2026, 4:01:33 pm
Rs. 10
COMPLETED
#1
A1 (CAR)
MH67TY6543
19/4/2026, 3:58:31 pm
19/4/2026, 4:03:36 pm
Rs. 20
COMPLETED


localhost:5174/history
Complete Payment
Please enter payment details to finalize your parking exit.
Vehicle:
MH65JH8900
Slot:
A5
Total Amount:
Rs. 20
Card Details (Simulated)
4242 4242 4242 4242
12/26
•••
Cancel
Pay Rs. 20





9. Conclusion

The Parking Management System successfully automates parking operations, reduces manual effort, and improves efficiency. The system ensures secure authentication, real-time tracking, and accurate billing. It demonstrates the effective use of modern web technologies like React, Spring Boot, and MySQL.

Future improvements:

- Online payment integration (Razorpay/Stripe)
- Real-time IoT sensor integration
- Mobile application

10. References

- Spring Boot Documentation
- React.js Documentation
- MySQL Documentation
- JWT Authentication Guide
- Stack Overflow.